Programmieren 2 – Portfolio 2

Christopher Engert

Technische Dokumentation

Da mir das Thema der objektorientierten Programmierung und die verschiedenen Sorten an Klassen in Kotlin anfangs etwas komplex erschienen, habe ich mir im Vorfeld des Portfolios von einer KI die genauen Anwendungsfälle der gelernten Klassenarten und Konstrukte in einer Art Cheatsheet zusammenfassen lassen. Die dort bereitgestellten praktischen Beispiele halfen mir, die theoretischen Konzepte vorab besser zu verinnerlichen.

Stufe 1: Grundlegendes Domänenmodell

In der ersten Stufe ging es darum, die Basisdaten für eine Transaktion sauber abzubilden. Statt einfach nur Standard-Datentypen wie String oder Double zu nutzen (was im Code schnell unübersichtlich wird und zu Fehlern führen kann), wurden eigene, feste Klassen für die Fachwerte eingeführt.

Folgende Entscheidungen habe ich getroffen:

1. Die Typen Money, Email und TransactionId wurden als Datenklassen angelegt. Da diese Objekte selbst keine eigenen Entitäten sind, sondern reine Datenträger für Werte, bietet sich dieser Typ an. (50€ sind inhaltlich das selbe wie 50€).
2. Um zu verhindern, dass Objekte mit völlig falschen Werten erstellt werden, wurde direkt in den init-Blöcken der Konstruktoren eine Validierung mit require eingebaut. Money verbietet negative Beträge, Email verlangt @ und einen Punkt, TransactionID darf nicht leer sein.
3. Die Hauptklasse Transaction führt die einzelnen Bausteine zusammen. Hier wurde im init-Block zusätzlich die Regel eingebaut, dass Absender und Empfänger nicht identisch sein dürfen, da man kein Geld an sich selbst senden soll.

Auf folgende Schwierigkeiten bin ich gestoßen:

1. Da ich im Bereich JUnit-Tests nicht sehr fit war, habe ich mir von einer KI die Möglichkeiten der Assertions erklären lassen.
2. Ich hatte Probleme mit meiner Gradle-Datei, auch hier durfte mir eine KI helfen.

Stufe 2: Polymorphie und Ergebnisse der Verarbeitung

In der zweiten Stufe wurde das System erweitert, damit Zahlungen über verschiedene Anbieter abgewickelt werden können, ohne dass die Hauptklasse Transaction den genauen Ablauf kennen muss.

Folgende Entscheidungen habe ich getroffen:

1. Es wurde das Interface PaymentStrategy erstellt. Die konkreten Zahlungsarten PaypalPayment und CreditcardPayment bauen auf diesem Interface auf und haben ihre eigenen Regeln (Limits).
2. Für die Rückmeldung einer Zahlung wurde die sealed class PaymentResult mit den Zuständen Success und Failure gebaut.

Auf folgende Schwierigkeiten bin ich gestoßen:

1. Beim Schreiben der Bedingungen in den Zahlungsklassen war der Zugriff auf den eigentlichen Geldbetrag anfangs eine kleine syntaktische Hürde. Da der Betrag in der Klasse Transaction als eigenes Objekt (amount: Money) hinterlegt ist, dieses intern aber wiederum die Variable amount: Double besitzt, lautet der Zugriff im Code transaction.amount.amount. Man muss sich hier also präzise Schritt für Schritt vom übergeordneten Objekt bis zum eigentlichen Zahlenwert durchhangeln, um die Limits korrekt prüfen zu können.

Stufe 3: Kontrollierte Objekterzeugung und Kapselung

In der letzten Stufe wurde die Architektur noch einmal stark umgebaut um das Problem der Exceptions aus Stufe 1 zu lösen und der Transaktion eine Art Zustandssystem zu geben.

Folgende Entscheidungen habe ich getroffen:

1. Die Konstruktoren von Money, Email, TransactionID wurden private gesetzt. Es ist von außen nicht mehr möglich die Objekte einfach so zu erstellen.
2. Stattdessen wird eine Fabrikmethode create() in einem companion object realisiert. Dies prüft die Daten vorab mit einer if Abfrage. Wenn die Daten nicht stimmen, wird NULL zurückgegeben.
3. Das Zustandssystem für den Lebenszyklus der Transaktion wurde ebenfalls durch eine sealed class (TransactionStatus mit Pending, Completed, Failed) umgesetzt. Der Grund dafür ist, dass wir im Zustand „Failed“ im Gegensatz zu einem normalen Enum eine dynamische Fehlerbegründung als Datenfeld mitgeben können. Zudem wurde die Variable über ein private set geschützt. Dadurch kann der Status zwar von außen gelesen, aber der Wert selbst nur über eine Methode in der Transaction Klasse geändert werden.

Auf folgende Schwerigkeiten bin ich gestoßen:

1. Die Tests spielten nach der Anpassung erst mal verrückt, da sie keinen Zugriff mehr auf Money hatten. Das sah zuerst erschreckend aus, allerdings konnte das schnell mit dem gewünschten .create und dem abschließenden !! gefixt werden.
2. Der alte Init-Block wurde an und für sich auf den ersten Blick unnötig und erschien mir als doppelter Code. Ich habe mich hier von einer KI beraten lassen, ob ich ihn problemlos löschen kann. Die KI klärte mich auf, dass es trotz doppelten Codes sinnvoll sein kann diesen zu behalten, da man trotz eines private Konstruktors über eine automatisch generierte .copy() Funktion von Datenklassen einen illegalen Wert einschleusen kann.